

PRISM — Architecture, Formalized via MBSA / RFLP

This restates PRISM's architecture using the same discipline as your MBSA reference material: architectural drivers first, then the RFLP perspectives (Requirements → Functional → Logical → Physical), then a traceability matrix — exactly the pattern used for the VMT reference example.

1. Architectural Drivers

Design Purpose: Currently exploratory prototyping (hackathon-stage codebase); target state is a real, low-cost production release with a Free/Pro tier. This dual purpose matters — it means the architecture must tolerate incomplete pieces now while not requiring a rewrite to reach production.

Architecturally Significant Requirements (ASRs):

ASR	Why it's architecturally significant
Integrity	The entire product's value proposition is trustworthy verification. A single fabricated "verified" source (as currently exists in <code>generateSandboxGNewsArticles()</code>) breaks the product's core promise, not just a feature.
Cost-boundedness	Solo, unfunded developer. Every architectural decision must keep marginal cost near-zero until usage (and revenue) justifies spending.
Modifiability	The LLM provider (Gemini today, possibly Claude/Groq tomorrow) must be swappable without touching business logic — you already do this partially with the candidate-model fallback array in <code>api/index.ts</code> .
Scalability under uncertainty	You don't know if 10 or 10,000 people will use this. The architecture must degrade gracefully (rate limits, caching) rather than fail or bankrupt you.

Constraints:

- Solo developer, part-time (student schedule)
- Free-tier infrastructure only, at least through MVP
- Existing codebase already committed to React 19 + Vite + Vercel + Firebase — architecture must extend this, not discard it
- No dedicated QA/ops — the architecture itself has to absorb some of that role (validation, monitoring)

2. RFLP Perspectives Applied to PRISM

Requirements Perspective

(Stakeholder needs, constraints, test scenarios — technology-independent)

- **Reader stakeholder need:** "Tell me if this article is trustworthy, and show me why."
- **Founder stakeholder need:** "Verify claims without spending money I don't have, and don't let the system say something false about itself."
- **Test scenario (integrity ASR):** *Given* no live news API key is configured, *when* the ingestion pipeline runs, *then* the system must show an explicit "no live data" state — it must never generate and label synthetic content as a verified source. (This directly targets the current `generateSandboxGNewsArticles()` defect.)
- **Test scenario (cost ASR):** *Given* 100 users request verification of the same trending story, *when* the second through hundredth requests arrive, *then* only one real LLM call should occur — the rest are served from cache.

Functional Perspective

(Technology-independent functions and data flows — the FAS layer)

Function	Data in	Data out
Ingest	Raw web sources (RSS/API)	Normalized article records
Deduplicate	Normalized articles	Unique story clusters
Score (classical)	Article text	Bias label, credibility estimate
Decide escalation	Classical score + user action	Route: basic / deep verification
Debate (deep verification)	Article + context	Multi-agent findings (progressive/conservative/omission/moderator)
Persist	All of the above	Stored, queryable records
Present	Stored records	Rendered feed, fact logs, source directory

This layer is intentionally technology-independent — it's the same regardless of whether "Score" runs on MobileBERT or a different classifier, or "Debate" runs on Gemini or Claude.

Logical Perspective

(Abstract component structure — this is what the diagram above shows)

- **User Web App** (logical component: presentation + local interaction state)
- **Central Cloud Service** (logical component: orchestration, auth enforcement, routing)
- **Verification Engine** (logical component, itself composed of two cohesive sub-blocks):
 - **Classical ML Scorer** — high cohesion: everything related to cheap, always-on scoring
 - **Tiered LLM Router** — high cohesion: everything related to provider selection, cost tiering, escalation
- **Storage** (logical component: persistence, currently under-specified — this is the gap to close)

Coupling is deliberately minimal: the Verification Engine doesn't need to know *how* the User Web App renders results, and the User Web App doesn't need to know *which* LLM answered — only whether it was a "basic" or "deep" verification. This separation is what lets you swap Gemini for Claude later without touching the frontend.

Physical Perspective

(Concrete deployment — hardware/software nodes and interfaces)

Logical component	Physical realization
User Web App	React 19 + Vite, deployed as static assets on Vercel
Central Cloud Service	Express app in <code>api/index.ts</code> , deployed as a Vercel serverless function
Classical ML Scorer	<code>bias.worker.ts</code> — ONNX runtime (MobileBERT + MiniLM) executing client-side, WebGPU/WASM
Tiered LLM Router	New <code>llmRouter.ts</code> module — calls Gemini (<code>@google/genai</code>) today, extendable to Claude/Groq endpoints
Storage	Firestore (via existing <code>firebase-admin</code>) — physical interface already authenticated via <code>requireAuth</code>
Cache	Upstash Redis (new) — physical interface between Central Cloud Service and Tiered LLM Router

3. Traceability Matrix

Mirroring the VMT example's structure exactly:

Stakeholder Goal	System Requirement	Functional Block	Physical Block
Trustworthy verification	Zero fabricated "verified" content; every claim traces to a real, fetchable source URL	Provenance Validation	Ingestion worker (new <code>api/cron/ingest.ts</code>) — replaces <code>generateSandboxGNewsArticles()</code>
Affordable at solo scale	Bulk scoring must run without per-call LLM cost	Classical Scoring	<code>bias.worker.ts</code> + <code>biasEngine.ts</code> (existing, client-side ONNX)
Deep verification on demand	Full multi-agent debate only for flagged/high-priority/user-requested stories	Escalation Decision + Debate	Tiered LLM Router → Gemini/Claude candidate chain in <code>api/index.ts</code>
Efficient at repeat queries	Identical queries must not re-trigger paid LLM calls	Caching	Upstash Redis, keyed by story hash
Auditable trust	Users must be able to see <i>how</i> a verdict was reached, not just the verdict	Transparency Presentation	<code>TransparencyFeed.tsx</code> , Multi-Agent Fact Logs panel
Durable state	Votes, articles, and verdicts must survive server restarts	Persistence	Firestore, replacing in-memory <code>votesStore</code>

4. What this formalization changes practically

Nothing about the underlying tech stack changes from the previous architecture document — but this exercise surfaces two things the informal version didn't make explicit:

1. **Coupling discipline** — the Logical perspective makes clear that the frontend should only ever know "basic vs. deep" verification, never which specific LLM produced a result. Enforce this at the API contract level (a `verificationTier` field, as already suggested in `types.ts`) so provider swaps stay cheap forever, not just today.

2. **Test scenarios are now explicit, not implied** — the two Requirements-perspective test scenarios above (no fabrication; cache before re-verifying) give you concrete acceptance criteria for Phase 0, rather than a vague "fix the trust issue" instruction.

This RFLP structure is also directly reusable for your IEEE paper's methodology section if you go the hybrid classical-ML + multi-agent-debate route — architecture diagrams backed by a traceability matrix are exactly the kind of rigor reviewers look for.